

PROJECT TECHNICAL GUIDE

MM Scholar

Code modules and whole-system explanation

Purpose

A concise guide to the runtime architecture, recommendation engine, data layer, model lifecycle, frontend, testing, and deployment.

System type - Server-rendered Flask web application

Core design - SQL eligibility filtering followed by kNN ranking

Catalogue - 690 eligibility rows, 87 scholarships, 30 destinations

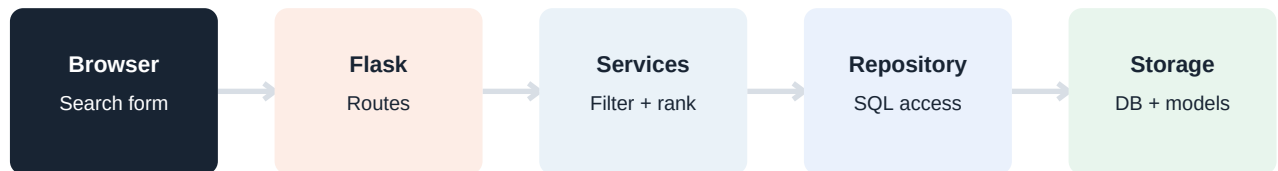
Production ranker - Distance-weighted kNN, $k=7$

Prepared for - Thesis defense, July 2026

1. Whole-system architecture

MM Scholar separates request handling, business logic, database access, model inference, and presentation. This makes each responsibility easier to test and explain.

REQUEST PATH Eligibility is deterministic; ranking is model-based; explanation is optional.



Startup sequence

- **app.py** creates Flask and configures logging.
- Scholarship and feedback repositories are created.
- The feedback table and indexes are initialized.
- Dropdown options, catalogue statistics, region counts, and model metrics are loaded once.
- The saved kNN pipeline and label encoders load when the recommendation service is imported.

Runtime boundaries

- **Routes** translate HTTP requests into service calls and templates.
- **Services** own validation, fallback filtering, ranking, and explanations.
- **Repositories** own SQL and database connections.
- **Templates** render the interface and trigger optional browser actions.

Key principle: the model does not decide basic eligibility. Explicit rules filter first; machine learning orders the remaining candidates.

2. Runtime code modules

Module	Responsibility
app.py	Flask setup, routes, startup data, admin authentication, errors, health endpoint
config.py	Paths, environment variables, database selection, validation limits, logging
services/recommendation_service.py	Profile validation, four-level SQL fallback, kNN scoring, top-three ranking
services/explanation_service.py	Gemini prompt, in-memory cache, student-focused explanation
repositories/scholarship_repo.py	Catalogue queries, recommendation candidates, pagination, statistics, admin operations
repositories/feedback_repo.py	SQLite/PostgreSQL feedback storage, editing, deletion, statistics, health check

Primary Flask routes

- **/** - search form, statistics, and recent feedback.
- **/recommend** - validates a profile and renders ranked results.
- **/explain** - returns an optional Gemini explanation as JSON.
- **/scholarships** - filtered catalogue with pagination.
- **/compare** - evaluation metrics.
- **/feedback** and **/feedback-results** - student feedback.
- **/health** - database availability.
- **/admin** - protected management dashboard.

Configuration and security

DATABASE_URL activates PostgreSQL for feedback; otherwise SQLite is used. GEMINI_API_KEY, ADMIN_PASSWORD, SECRET_KEY, PORT, and FLASK_DEBUG are environment-controlled. Admin username is fixed as **admin**. The development secret-key fallback must be replaced in production.

3. Recommendation engine

The recommendation service combines input validation, deterministic candidate filtering, and model ranking.

Step 1 - Validate the profile

- Country, study level, and funding type must be present.
- GPA must be numeric and between 0 and 4.
- IELTS must be numeric and between 0 and 9.
- Blank GPA or IELTS values are optional.

Step 2 - SQL fallback chain

Stage	SQL constraints	Label
1	Country + level + funding + optional field/GPA/IELTS	exact
2	Remove funding preference	relaxed_funding
3	Keep country and academic thresholds	country_only
4	Fully funded + requested level + thresholds	popular_fallback

Step 3 - Model score and shortlist

```
student profile -> one-hot categories + scaled GPA/IELTS  
-> kNN predict_proba  
-> retain SQL candidate scores  
-> sort -> deduplicate -> top 3
```

The model produces scores across all modeled scholarship classes. The service maps SQL candidate names to encoded classes, retains those scores, sorts them, and returns three unique scholarships.

Score interpretation: match_pct is a relative distance-weighted class score. It is not a calibrated probability of eligibility, admission, or receiving an award.

Failure behavior

If prediction fails, the service returns up to three unique SQL-filtered scholarships with match_pct set to None. The UI hides the score instead of presenting a fabricated zero.

4. Data and machine-learning lifecycle

Dataset

- The CSV is validated and converted into the scholarships table in mm_scholar.db.
- Each row is one valid scholarship, level, field, funding, and threshold combination.
- The catalogue contains 690 rows, 87 scholarship names, and 30 destination categories.
- The six production features are country, level, field, funding, minimum GPA, and minimum IELTS.

Training pipeline

- Classes with only one row are excluded from supervised training, leaving 84 modeled classes.
- An 80/20 split uses random_state=42 and stratification by scholarship class.
- kNN uses OneHotEncoder for four categorical features and MinMaxScaler for GPA and IELTS.
- Grid search evaluates k values 3, 5, 7, 9, 11, and 15 with Euclidean and Manhattan distance.
- The selected production pipeline uses k=7 and distance-weighted voting.
- Decision Tree, Random Forest, and Gradient Boosting are saved comparison baselines.

Evaluation results

Model	Role	Top-1	Top-3	CV mean
kNN	Production ranker	54.35%	74.64%	68.11%
Decision Tree	Baseline	81.16%	90.58%	84.72%
Random Forest	Baseline	79.71%	98.55%	82.97%
Gradient Boosting	Baseline	80.43%	97.83%	83.56%

Decision Tree has the best Top-1 result and Random Forest has the best Top-3 result. kNN remains deployed because the runtime problem is similarity ranking after deterministic filtering, not unrestricted hard-class prediction.

Artifact files

models/knn_model.pkl contains the complete production preprocessing and classifier pipeline. encoders.pkl maps scholarship names and baseline categories. The DT, RF, and GB files are retained for comparison. metrics.json drives the homepage and comparison page.

5. Data access, frontend, and explanations

Scholarship repository

- Fetches recommendation candidates with parameterized SQL.
- Returns catalogue rows grouped by unique scholarship name.
- Provides filters, pagination, dropdown values, regional counts, and project statistics.
- Supports admin listing, row deletion, and database health checks.

Feedback repository

- Uses PostgreSQL when DATABASE_URL exists; otherwise uses SQLite.
- Creates the feedback table and timestamp/rating indexes at startup.
- Stores search profile, model name, recommendation, rating, comment, and timestamp.
- Supports public summaries and protected admin edit/delete operations.

Jinja templates and browser behavior

Template	Purpose
index.html	Search form, project metrics, regional counts, recent feedback
results.html	Ranked cards, match score, official link, share controls, explanation, feedback form
browse.html	Catalogue filters and 12-item pagination
compare.html	Model evaluation comparison
about.html	Project and dataset explanation
admin.html	Feedback and scholarship administration
400/404/500.html	User-friendly error handling

Gemini explanation service

The explanation service receives a selected scholarship and student profile, creates a prompt with three short sections, and caches the response in memory. Gemini never filters or ranks scholarships. If it fails, /explain returns a safe fallback and the core recommendation remains available.

6. End-to-end request flow

The following sequence describes one complete recommendation request from the browser to feedback storage.

- **1. Search:** the student submits country, level, funding, field, GPA, and IELTS from index.html.
- **2. Route:** POST /recommend reads the form and calls get_recommendations.
- **3. Validation:** required fields and numeric ranges are checked.
- **4. Candidate selection:** ScholarshipRepository executes the SQL fallback chain.
- **5. Feature construction:** the service creates one six-feature student DataFrame.
- **6. Inference:** the saved kNN pipeline preprocesses the profile and returns class scores.
- **7. Candidate ranking:** only SQL candidate classes are retained, sorted, deduplicated, and limited to three.
- **8. Presentation:** results.html shows scholarship details, official links, and any valid match scores.
- **9. Optional explanation:** browser JavaScript calls POST /explain for one selected result.
- **10. Feedback:** the student rating is saved through FeedbackRepository.

Operational scripts

- convert_to_sqlite.py validates the CSV and rebuilds the indexed SQLite catalogue.
- train_models.py tunes and saves production and baseline pipelines.
- evaluate_models.py calculates metrics, cross-validation, robustness checks, and charts.
- eda.py and notebooks/eda.ipynb document dataset exploration.
- export_feedback.py and import_feedback.py back up and restore PostgreSQL feedback.
- test_app.py runs 86 project checks.

Deployment

```
Procfile: web: gunicorn app:app
Local run: python app.py
Health check: GET /health
```

Defense summary: MM Scholar is a hybrid decision-support system. SQL preserves explicit eligibility logic, kNN supplies similarity ranking, Jinja provides a transparent interface, Gemini is optional, and official scholarship sites remain the final authority.